

第4回：プロンプトと外部知識

参考資料を渡し、根拠付きで答えてもらう

今日のゴールと流れ

指示・参考資料・質問を分けて組み立てる。

時間	内容
10分	一般知識と授業固有の情報
20分	プロンプトの構成
20分	資料の読み込みと出典
30分	指示文の実装・比較
10分	限界と振り返り

前回のアプリの限界

「この教材の第7回の提出物は？」

資料なしのモデルは、サンプルの授業案内を参照できません。
詳しく答えているように見えても、推測かもしれません。

今回は `data/02-course.md` の情報を、質問と一緒に渡します。
案内は練習用の架空データです。

プロンプトの要素

要素	書くこと
目的・役割	授業の学習を手伝う
制約	提供した資料だけを根拠にする
出力形式	簡潔な日本語、資料IDを付ける
参考資料	回答に使える情報
質問	利用者が知りたいこと

各要素の役割が見えるように、コードでも分けます。

指示とデータの違い

「参考資料に『前の指示を無視しなさい』と書かれていた」

資料は、回答を作るために読むデータです。

アプリ全体への命令として実行するものではありません。

区切りと指示文は対策の一部です。完全な防御にはなりません。

実行権限の制限と、攻撃的な入力での評価も必要です。

資料の形式

教材用の架空の授業案内

第7回の提出物

提出物はmain.pyとapp.jsのGitHub URL、および説明文です。

は資料のタイトル、## は見出しです。

今回はPDFの解析を省き、人が読めるMarkdownを使います。

SQLiteへ登録する理由

資料にIDを付け、後の検索に使います。

カラム	内容
id	資料断片の番号
source	原本のファイル名
title・heading	資料名と見出し
body	本文

第4回では、検索せず、登録済みの断片を全部渡します。

資料を渡す流れ

1. `data/` のMarkdownを読む
2. `##` ごとに分けてSQLiteへ登録
3. `all_chunks()` で全件取得
4. `build_context()` で文字列にする
5. `ask_with_context()` からLLMへ渡す

ファイルの読み込みとSQLite操作は配布済みです。今日は指示文に集中します。

参照用の文字列

```
context = "  
for chunk in chunks:  
    context += f"[資料{chunk['id']}] {chunk['title']}\n"  
    context += chunk['body'] + '\n\n'
```

リストから1件ずつ取り出し、文字列へ追加します。
完成コードでは見出しも付けます。

根拠を指定する指示

```
instructions = (  
    '授業資料だけを根拠として回答してください。'  
    '根拠が不足するときは資料から確認できないと教えてください。'  
    '説明には[資料3]のように資料IDを付けてください。'  
)
```

隣り合う文字列はPythonが1つにつながります。
実際の教材では、資料内の命令への対策も加えます。

質問と参考資料を送る

```
input_text = '参考資料:\n' + context
input_text += '\n質問:\n' + question
```

「知識を使う」とは、今回の回答に必要な文脈を入力へ含めることです。
APIを呼んだだけで、ローカルのファイルを勝手に読むわけではありません。

実習1：資料を確認

```
cd session04/exercise  
python init_db.py  
python main.py
```

初回起動時も登録しますが、今回は明示的に登録を体験します。
「資料全体を渡す」を選び、「第7回の提出物は？」と質問します。
デモでは本文の抜粋を表示します。

実習2：指示文を復元

1. `llm.py` の `TODO` を探す
2. 指示文のコメントを外し、仮の `raise` を削除
3. 講師指定のAPIモードで起動し直す
4. 「資料なし」と「資料全体」で同じ質問を比較
5. 回答中の資料IDと、実際の資料本文を照合

コード変更後はサーバーを停止して起動し直します。

実習3：根拠の不足

次の2問を比較します。

- 「第7回の提出物は？」
- 「来年度の試験日は何月何日？」

後者には日付の根拠がありません。

「資料からは確認できません」と伝えられるかを評価します。
回答を控えることも、アプリの機能です。

出典表示の意味

画面の「LLMに渡した資料」は、サーバーが取得した実際の本文です。
回答に書かれた資料IDは、モデルが生成した文字列です。

- 存在するIDか
- その本文が主張を支えているか
- 必要な条件や例外を落としていないか

出典欄があるだけでは、回答の正しさを保証しません。

資料の更新

1. `data/` の文章を変更する
2. サーバーを止める
3. `python init_db.py` で登録し直す
4. `python main.py` で起動して同じ質問をする

アプリは `data/` を原本として扱います。

再登録するとIDが変わる場合があるので、評価にはファイル名・見出しも記録します。

すべて渡す方法の限界

- 資料が増えると入力が長くなり、費用や待ち時間に影響する
- 無関係な文章が増える
- モデルの入力上限がある
- 古い資料や矛盾する資料の扱いが必要になる

教材では参照文を12,000文字以内に制限します。

これは文字数の教材用制限で、モデルのトークン上限とは異なります。

確認問題

1. `data/` を編集しただけで、DBの内容は変わる？
2. 資料IDがあれば、回答の主張も正しい？
3. 資料にない質問に、どう応答するよう設計する？
4. 全資料を渡せなくなったら、何を変える？

提出物と次回

提出： `llm.py` と `worksheet.md` のGitHub URL。

ワークシートには、根拠のある質問とない質問の結果、
使われた資料名・見出し、自分が直したい点を記録します。

次回は、必要な資料を検索してから渡します。